# TRANSFER LEARNING TECHNIQUES TO SOLVE TRANSIENT NON-LINEAR PARTIAL DIFFERENTIAL EQUATIONS

A Thesis
Submitted for the Partial Fulfilment of the Requirements for the Degree of

**BACHELOR OF TECHNOLOGY**

in

**Mechanical Engineering**

submitted by

Bulusu Sri Datta Sudheer
18ME01017

under the guidance of

Dr. B Pattabhi Ramaiah


School of Mechanical Sciences
Indian Institute of Technology Bhubaneswar

May 2022

# CERTIFICATE

It is certified that the work contained in the thesis titled **Transfer Learning approaches to solve transient non-linear partial differential equations**, is a bonafide work of **Bulusu Sri Datta Sudheer**, has been carried out under my supervision for the award of degree of B.Tech. in Mechanical Engineering, at School of Mechanical Sciences, Indian Institute of Technology Bhubaneswar.

This reported work is carried out during the academic year 2021- 2022 and has not been submitted elsewhere for a degree.

Date:

Dr. B Pattabhi Ramaiah
Assistant Professor
School of Mechanical Sciences
Indian Institute of Technology, Bhubaneswar
Bhubaneswar, 752050, Odisha, India

# ACKNOWLEDGEMENT

I want to express my sincere gratitude to my supervisor Dr. Pattabhi Ramaiah, School of Mechanical Sciences, IIT Bhubaneswar, for providing me with the opportunity to work under their esteemed supervision on this research project. Their invaluable pieces of advice during the meetings for discussing the ongoing research project provided great insights and direction for its progress.

B Sri datta Sudheer

# DECLARATION

I certify that,

1. The work contained in this thesis is original and has been done by me under the guidance and supervision of Dr. Pattabhi Ramaiah.
2. The work has not been submitted to any other institute for any Degree or Diploma.
3. I have followed the guidelines stipulated by the institute in preparing the Thesis.
4. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the institute.
5. Whenever I have used materials (data, theoretical analysis and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the References.

Bulusu Sri Datta Sudheer

18ME01017

# ABSTRACT

In science and engineering, Partial Differential Equations are encountered most frequently, and their solutions are crucial in many of the applications. But almost all Partial Differential Equations are hard to solve analytically, hence several numerical solvers have been developed which can solve the Partial Differential Equations using different methods such as Finite Element methods. Despite these developments, the high-performance numerical solvers may take long durations (days or weeks) to solve very sophisticated problems. So, the design of efficient algorithms which can solve the partial differential equations are necessary. Recently, Physics Informed Neural Networks, a framework has been formulated based on Artificial Neural Networks, which is numerically more efficient than the Finite Element based numerical solvers.

Car crash analysis is one of the main areas of research in automotive industries and solving it traditionally by wrecking the car to the wall physically causes a lot of financial loss and solving it using Finite Element methods consumes a lot of time. In this context, the problem of car crash has been taken and a comparison between the solutions obtained using Physics Informed Neural Networks framework and Finite Element based numerical solvers have been made. Also, exploring how the accuracy can be even improved using the techniques of Transfer Learning.

Keywords: Car crash, Artificial Neural Networks, Physics Informed Neural Networks, Partial Differential Equations, Finite Element Analysis

# CONTENTS

# LIST OF FIGURES

# 1. INTRODUCTION

Partial Differential Equations (PDEs) play a significant role in many disciplines of science and engineering to describe the governing physical laws involved in a given problem of interest. They are commonly derived based upon principles of physics such as conservation laws, energy principles or based on empirical observations. Popular examples include the Navier-Stokes equation, maxwell equations, etc. Solving Partial Differential Equations involved in the typical problems is generally analysed and solved using FEA (Finite Element Analysis). It is the process in which we can simulate the behavior of a part or an assembly under certain conditions. It uses numerical methods to solve PDEs involved. Coming to how FEA works, it divides a larger problem into a large no of smaller and simpler parts called finite elements. Each smaller element is subjected to the given conditions. For example, for finding the stress in a bar, the bar is divided into a large no of smaller elements, the stiffness matrix at every finite element is calculated and, combined it to form a global stiffness matrix and then the equation K.U = F is solved to obtain displacements and strain. From young's modulus the stress in a bar can be obtained.

The problems faced when using FEA are:

- Requires high computational power, time and computer memory in order to obtain the result.
- Uses the displacement function which is approximated to linear or any higher order differential equation.
- Involves finding the stiffness matrix at element level and they are assembled to obtain global stiffness matrix.

*"We can use augmented simulation to speed up the simulation by factors of 100X by training neural networks via data-driven or physics-informed methods." - Ansys*

Neural network is a type of network inspired by the arrangement of neurons inside brain. Connections are modelled as several layers, each layer containing many nodes(neurons). Each layer extracts some features from the data and predicts the outcome inter-connecting those features.As stated in an Ansys article which says that neural networks can be used to increase the speed of the simulation. PINN (Physics Informed Neural Networks) are neural networks which uses PDEs as a part of loss function so that it determines the solution by following laws of physics as in PDEs hence the name Physics Informed. The idea comes from a set of papers by Lagaris et al[2], on PINNs published in 1990. To solve the simulation problems, PINN has to be trained at each node. This is also a kind of repetitive task we can computationally optimize it using transfer learning.

Transfer learning is a kind of machine learning technique in which we use the model developed for a particular task as a starting point to another task. For example, if we have the model of image recognition of car, we can apply it to image recognition of trucks. For cars, the model identifies tires, headlights, windshield of the car, the same things are also present in trucks.

## 2. OBJECTIVES

1. To determine crash displacement, velocity profile, stress profile involved when a car is crashed by hitting a full-frontal barrier using PINN.

2. To compare the solution of crash displacement of a car when it is crashed by a full-frontal barrier obtained using PINN to that of the solution obtained using Finite Element Analysis.

3. To perform transfer learning by considering the solution of car crash of a model of a car to different models of cars.

## 3. BACKGROUND

Deep Learning and Neural Networks:

Suppose that we have given a labelled dataset of N data points. The goal of training an ANN is to determine its parameters $W$ such that it approximates the mapping inherent to as good as possible with respect to some metric $L$ called loss function.

$$W^* = min_w\, L(\boldsymbol{y}, \widehat{u}(x))$$

Where, $\widehat{u}(x)$ is the network's prediction and **y** are the label to be approximated. The most used and most popular loss function is the mean squared loss, given by

$$L\big(y, \widehat{u}(x)\big) = \frac{1}{N} \sum (\boldsymbol{y}_i - \widehat{u}(x))^2$$

Universal Approximation Theorem:

A fully connected ANN can approximate any function to a greater accuracy. In other words, ANNs are nothing but function approximations. But in reality, limits may be reached due to insufficient capacity of approximation for very small networks, hence too few parameters and a low dimensional $W$, and convergence issues in the weights optimization task. The property of universal function approximation property of ANNs justifies their use for approximating any quantity of interest.

Physics Informed Neural Networks:

As already mentioned. Neural networks based on PINN framework, use PDE to formulate the loss function of the neural network, so they are trained to solve supervised learning tasks while respecting the laws of physics.

The interesting fact is the formulation of the neural network as, unlike the traditional ways, there is not a clear training set, testing set or a validation set This is a clever way of posing a problem in which the ODEs or PDEs are turned into simple optimization problem.

Let's see this with an example,

$\frac{d^2u}{dx^2} + a\frac{du}{dx} = b$ Boundary conditions: $u(0) = u_0$, $u(1) = u_1$

By universal approximation theorem, one can always approximate the solution of u arbitrarily closely by neural network which states that we can write u as output of a neural network with x as input.

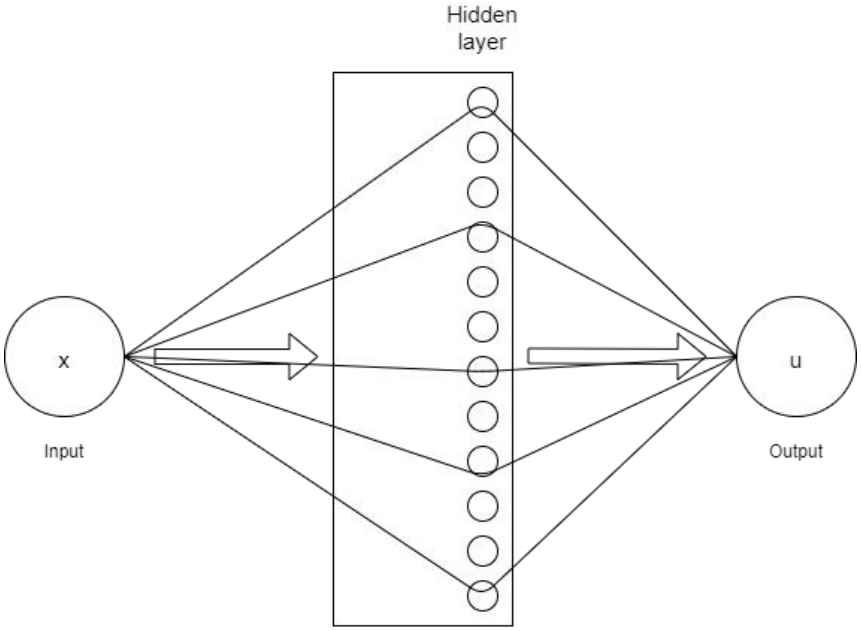


Fig 3.1 *Representing a simple Neural network with 1 hidden layer of neurons*

A single layer neural network is depicted as above with some neurons in the layer. For simplicity let's take a single layer, single neuron network. Let's take the activation function as sigmoid. Activation function is a non-linear function which determines how the weighted input is related to the output from a node in a layer of network. There are many other activation functions like ReLu, leaky ReLu etc.

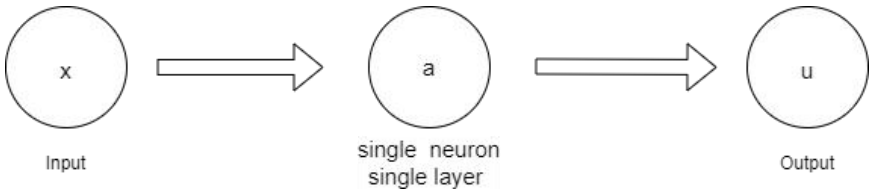


Fig 3.2 *Representing a simple Neural network with 1 layer and 1 neuron*

From the simple network depicted above, we can write a as sigma times weighted input, i.e., $w_1$*x or $w_2$*x

$$a = \sigma(W_1 x) \qquad u = W_2\sigma(W_1 x) \qquad \frac{du}{dx} = W_2\sigma'(W_1 x)W_1$$

similarly other higher order differentials can also be found. Even if there are multiple layers and multiple neurons involved, $\frac{d(output)}{d(input)}$ can be found using backprop. TensorFlow also has some inbuilt functions.

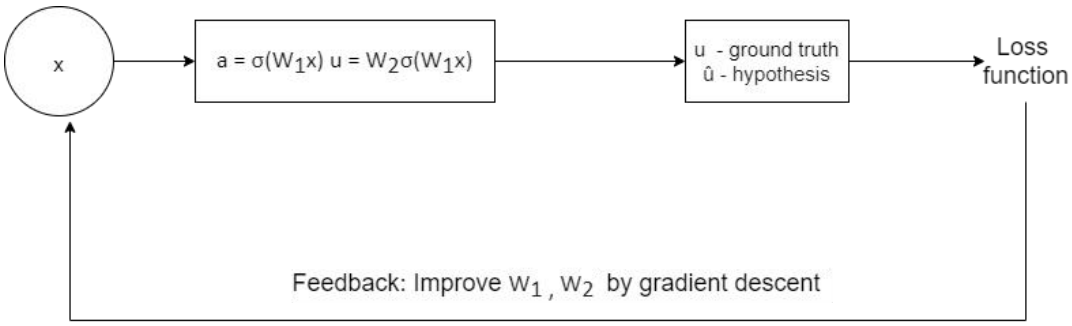


Fig 3.3 *Representing process of updation of the hypothesis using gradient descent*

Let's say we know the ground truth u, and the output of the neural network be $\hat{u}$ which is the hypothesis. Now we minimize the loss and improve $\hat{u}$ we recompute the weights using gradient descent. This process is called backprop whereas finding the hypothesis is called forward prop. Now we use PDE to determine loss function as this is PINN which we have discussed earlier. We minimize the square of this expression which is actually zero the minimum value of this expression is also zero thereby ensuring the solution to be close to exact. We also use boundary conditions in the loss function, we minimize the loss function and this means the min value of each of the individual terms which is zero and which is supposed to be the exact solution. While minimizing the loss function, we can obtain the derivatives of output with respect to input. So, the process is, first we take some arbitrary weights find the hypothesis minimize the loss function compute new weights while minimizing and then again find the hypothesis.
Now this can be applied to PDEs as well.

From Raissi et al.[1], the PDE and its boundary valued problems have been formulated as optimization problems. The idea is to utilize the governing equations of the PDE to convert them into optimization problems with loss function,

$$L = L_{de} + w_i L_i + w_b L_b$$

Where $L_{de}$ is Loss from Differential Equation, $L_i$ is Loss from Initial Condition, $L_b$ from Boundary Conditions and $w_i$ , $w_b$ are weighting scalars to account for the relative importance of the loss term.

# 4. FORMULATION

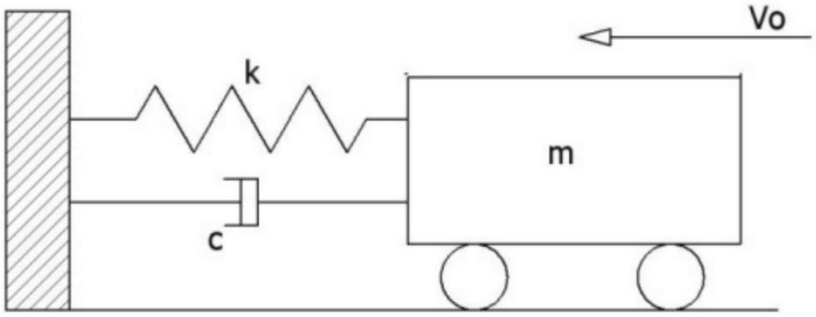


Fig 4.1 *1 DOF Spring mass damper system representing mathematical model of a car crash*

The model in Fig 4.1 has been proposed as a mathematical model to the vehicle in a vehicle to a fixed barrier collision.
In underdamped conditions, the equation of motion is: $\ddot{\alpha} + 2\zeta\omega_e\dot{\alpha} + \omega^2\alpha = 0$

Where, $\zeta = \frac{c}{2m\omega_e}$ and $\omega_e = \sqrt{\frac{k}{m}}$

The author used matlab simulink software to validate the transient responses.
The author have defined teo time periods:
$t_m$– time of dynamic crush and $t_f$ – time of rebound (time of separation velocity).
At $t_m$, the corresponding velocity is zero and the dynamic crush reaches its maximum value. At $t_f$, the corresponding deceleration is zero and velocity reaches its maximum value.

Using the relations of $t_m$ and $t_f$,the structural parameters k and c are deduced.[2]

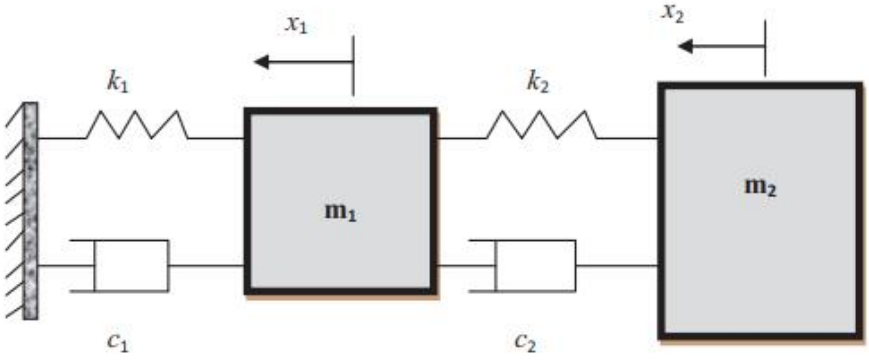


Fig 4.2 *2 DOF Spring mass damper system representing mathematical model of a car crash*

During the full-frontal (head-on collision) car crash, the vehicle is impacted by the obstacles. The model in Fig 4.2 simulates the efffect of a rigid barrier impact on a vehicle where $m_1$ and $m_2$ represent chassis and passenger compartments respectively.

The parameters which are to be estimated are spring stiffness constants $k_1$ and $k_2$, damping constants $c_1$ and $c_2$, as shown in Figure 4.2. When a vehicle is impacted by a rigid barrier during the full-frontal crash, the two masses will experience an impulsive force during collision. The method of solving the impact response of the two masses is similar to the method used in the free vibration analysis of a two degrees of freedom damped system.

The dynamic equations of the two mass-spring-damper model are shown in Equation.

$$m_1 \ddot{x_1} + (c_1 + c_2)\dot{x_1} + (k_1 + k_2)x_1 - c_2\dot{x_2} - k_2x_2 = 0$$

$$m_2 \ddot{x_2} - c_2\dot{x_1} + c_2\dot{x_2} + k_2x_2 - k_2x_1 = 0$$

In [2], The predefined shapes of the spring stiffness and damper constants are selected based on the shape of the displacement-time and velocity-time relations during the crash test. Maximum displacement occurs when the vehicle speed is zero. During the breaking phase, where the vehicle is overdamped and undamped during low and high velocities respectively. This indicates that the damping coefficient value at the time of maximum crash is high and the damping coefficient value at the initial velocity is low at the initial velocity. The spring stiffness is low during elastic deformation (obeying hookes law), but after crash the vehicle is plastically deformed, therefore the stiffness increases drastically to maintain the deformation.

The values of the parameters, spring stiffness constants $k_1$ and $k_2$, damper constants $c_1$ and $c_2$ can be directly taken from [2] which are found using the approach described above.

For 1 dof system, the PDE residual for PINN will be,

$$L = (m\ddot{x} + c\dot{x} + kx - 0)^2$$

For 2 dof system, the PDE residual for PINN will be,

$$L = (m_1 \ddot{x_1} + (c_1 + c_2)\dot{x_1} + (k_1 + k_2)x_1 - c_2\dot{x_2} - k_2x_2)^2 + (m_2 \ddot{x_2} - c_2\dot{x_1} + c_2\dot{x_2} + k_2x_2 - k_2x_1)^2$$

Weights corresponding to initial value conditions are considered as unity.

To solve the PDE using the Physics Informed Neural Networks as described above, the following steps can be followed.

Step 1 – Import necessary packages and set problem specific data.

The code runs with TensorFlow v2.3.0. The implementation relies mainly on the computing library Numpy and the machine learning library TensorFlow.

Problem specific data includes the defining initial conditions of the PDE, defining boundary conditions of the PDE, defining the residual of the PDE and also setting the constants obtained from analytical solution in [2].

Here, the initial condition is that, both the springs are initially taut and the residual is formed using the equations of motion of the mathematical model.

Step 2 – Generate a set of collocation points.

Collocation points in the time domain and points of initial value condition are generated by random sampling from a uniform distribution.
The chosen training data (collocation points) is of size 10000. And for initial data, it is 50.
The boundaries of the inputs are determined and the training data is found by using random sampling functions of TensorFlow.

Step 3 – Set up network architecture.

In this implementation, adopted from Raissi et al., [1], we consider a feed forward neural network of the following structure.

- The input is scaled to lie between the specified limits.
- 4 hidden layers with 12 neurons each is present after the input, with a hyperbolic tangent activation function after each of the layer.
- One fully connected output layer.

This setting of network architecture results in a network with 674 trainable parameters (first hidden layer: 1*12 + 12, four intermediate layers each: 12*12 + 12, output layer: 12*2 + 2)

Step 4 – Define loss and gradient.

Using the automatic differentiation capabilities of TensorFlow, necessary partial derivatives can be found using which gradient can be computed.

Loss function can be found using the PDEs, initial and boundary conditions as discussed earlier.

Step 5 – Set up optimizer and train model.

In this step, we initialize the model, set learning rate and an optimizer to train the model.
The optimizer used in this implementation is Adam optimizer.

The adam optimizer combines two concepts:
1. Momentum: Accumulate gradients over time to incorporate an exponentially-weighted average of gradient.
2. Damp osciallations for high variance directions.

Initialization is also of prime importance. In this project, Xavier/Glorot Initialization technique was used. The technique is to ensure that the variances of input and output are in similar range, leading to smooth gradient updates. To do so, all bias vectors are initialized as zero vectors.

In the task of transfer learning, the initialization is done by utilizing the solution of the source model.

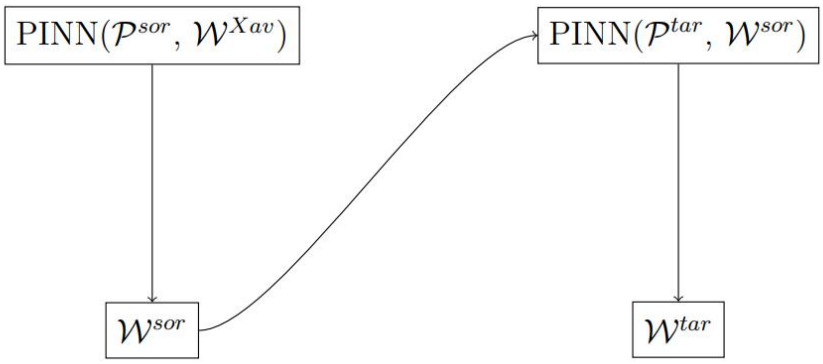


Fig 4.3 *Depicting the transfer learning process using PINN source model.*

The weights obtained after training for the source problem is directly utilized at the initial stage for the target problem.

## 5. RESULTS AND DISCUSSION

Using the PINN approach, the PDE obtained from the analytical method with suitable parameters is solved.

For 1 DOF model, the following result is obtained.

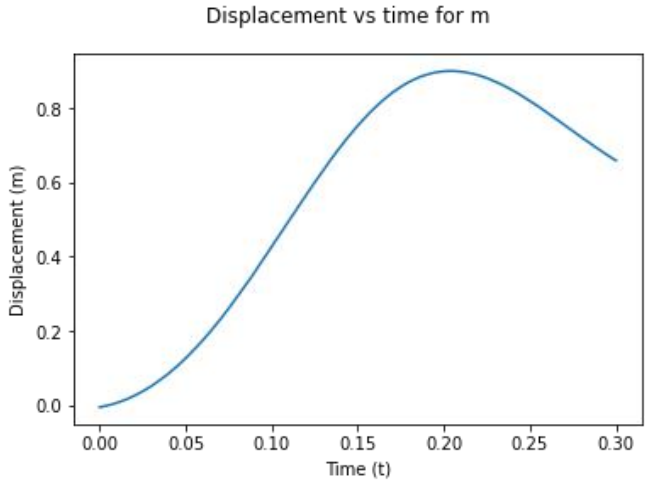


Fig 5.1 *Deformation in spring of 1 DOF model solved using PINN*

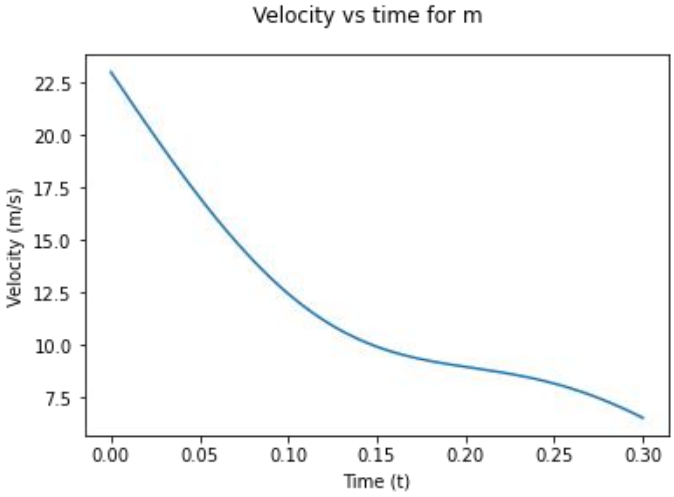


Fig 5.2 *Velocity response of 1 DOF solved using PINN*

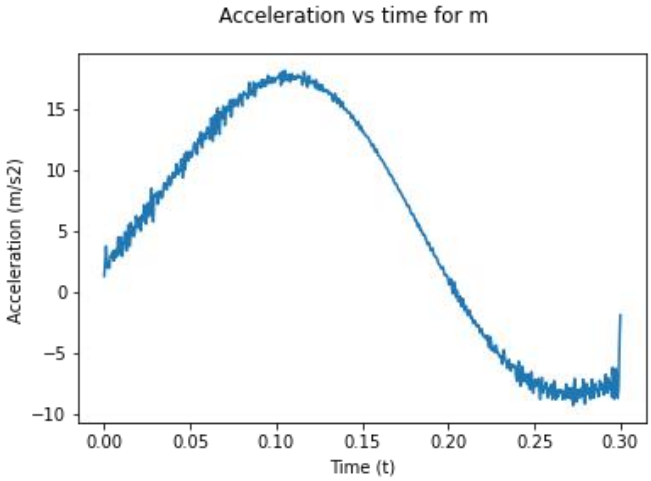


Fig 5.3 *Acceleration response of 1 DOF solved using PINN*

The displacement, velocity and acceleration responses of 1 DOFanalytical model solved using PINN are shown above. The displacement response shows a large increase until a certain time which is the maximum dynamic crash, after the maximum dynamic crush the car moves backwards due to the impulse. Hence, it decreases. While the velocity decreases with time.

For 2 DOF model, the following result is obtained.

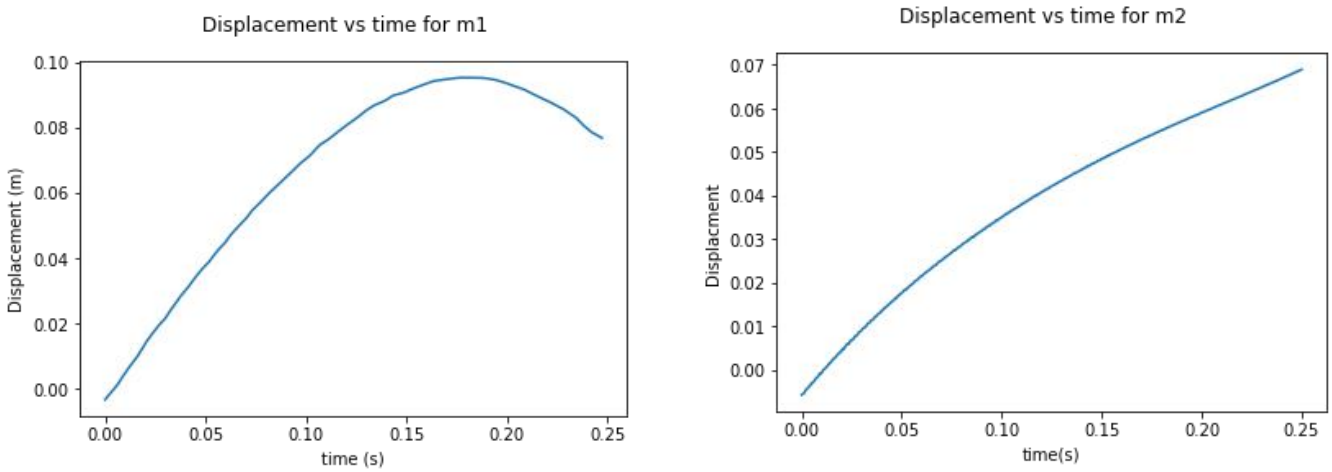


Fig 5.4 *Deformation in each spring of the model solved using PINN*

Fig 5.4 represents the deformation of each spring as a function of time, of the analytical model which is a 2 DOF spring mass damper system, solved using PINN methods.

From the solution obtained in [2], it is observed that maximum deformation is 80.62 cm and is obtained at nearly t = 0.17s.

Deformation solution from PINN methods is 0.95m at nearly 0.17s and 0.70m at nearly 0.25s. The displacement, velocity and acceleration responses of 1 DOFanalytical model solved using PINN are shown above. The displacement response shows a large increase until a certain time which is the maximum dynamic crash, after the maximum dynamic crush the car moves backwards due to the impulse. Hence, it decreases. While the velocity decreases with time.

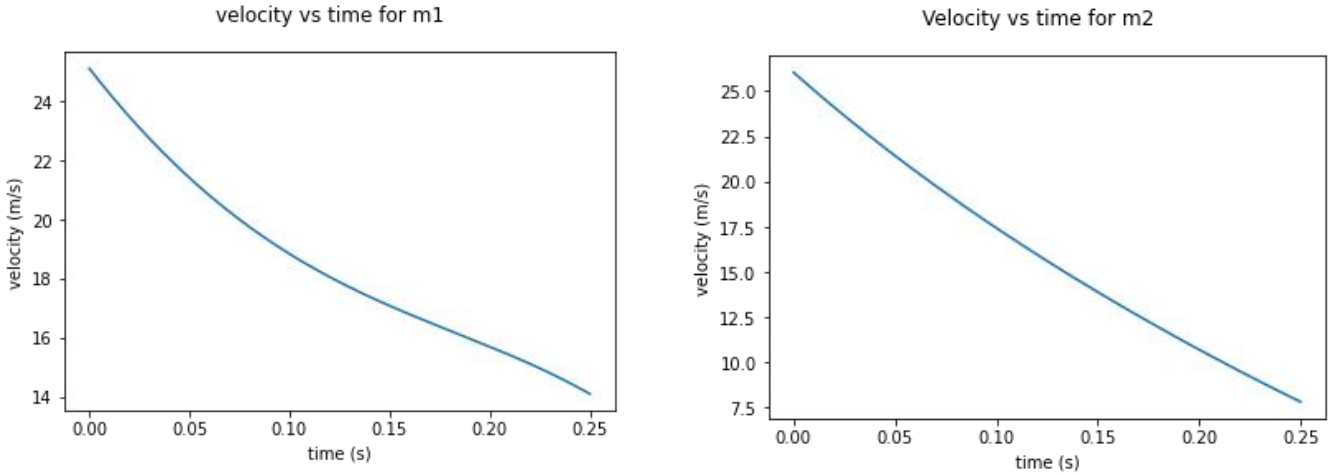


Fig 5.6 *Velocity response of the car crash obtained from PINN*

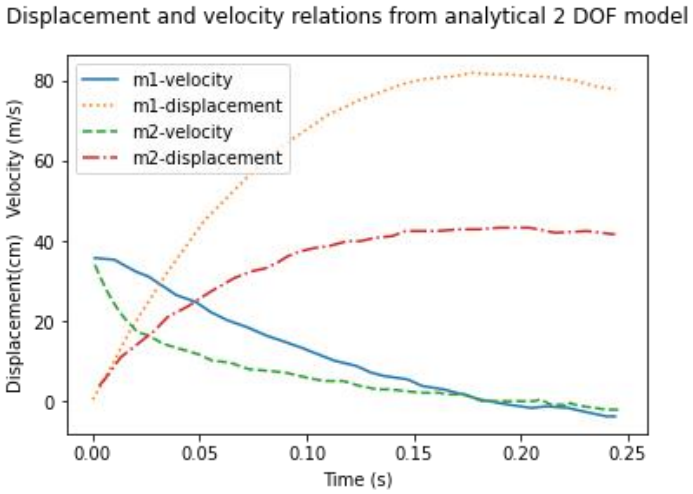


Fig 5.5 *Analytical solution of the car crash obtained from [2]*

From the velocity response of the car crash obtained from PINN methods, we can observe that velocity is decreasing from an initial value to zero as the time progresses which is also the case in the analytical solution of the crash as stated in [2]

The weights of the solution obtained in 2 DOF are taken as initial guess for the new problem with different structural parameters of the 2 DOF analytical model. The results obtained are:

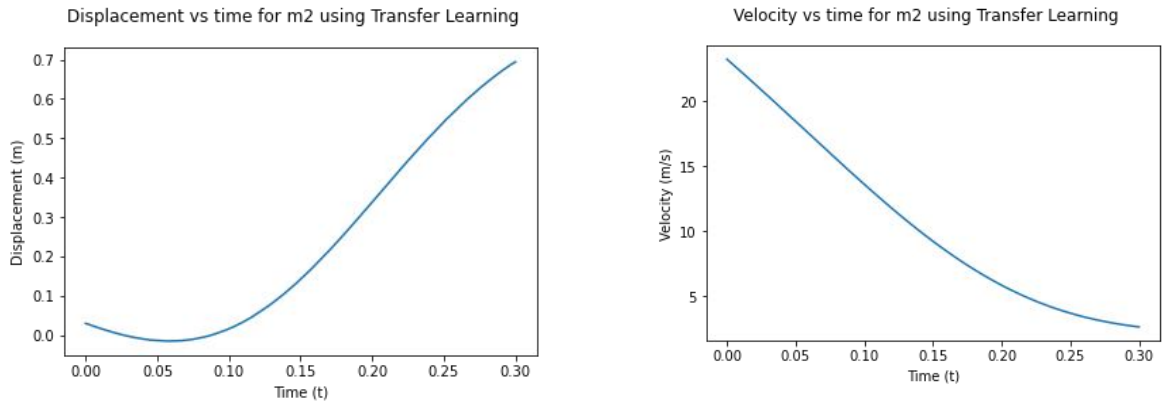


Fig 5.7 *Displacement and velocity responses for 2 DOF model using transfer learning*

The results from transfer learning are shown in fig 5.7. The behavior of the velocity plot is similar as the previous results from the car crash, but the behavior of the displacement plot varies significantly. The expected trend is to increase until a certain time called time of maximum dynamic crash and then decrease continuously. But, the trend obtained is completely different as it decreased at first for a very short time and then continuously increased. The variation in the behavior is mainly because of the inappropriate solution of the source from which the weights are taken as an initial guess for the transfer learning problem. This can be corrected by a suitable analytical model with well imposed boundary conditions.

The result obtained above are validated using the 1st approach as discussed earlier i.e., utilizing CAE solvers using FEA.

CAE solver used here is ANSYS and ANSYS explicit dynamics engineering simulation solutions are ideal for these types of physical events which occur for a very shorter duration of time. Explicit dynamics analysis is often used to determine the dynamic response of a structure due to stress wave propagation, impact or rapidly changing time dependent loads.

Solution strategy:

In an Explicit Dynamics solution, the model preparation steps are: a discretized domain (mesh), assigned material properties, loads, constraints and initial conditions. This initial state, when integrated in time, will produce motion at the node points in the mesh.

The model involves a car body and a wall.
Car body is assigned aluminium alloy and wall is assigned concrete material.
The Johnson cook damage and failure parameters are plugged into the engineering data section.
Mass of the car body is nearly 850kg.

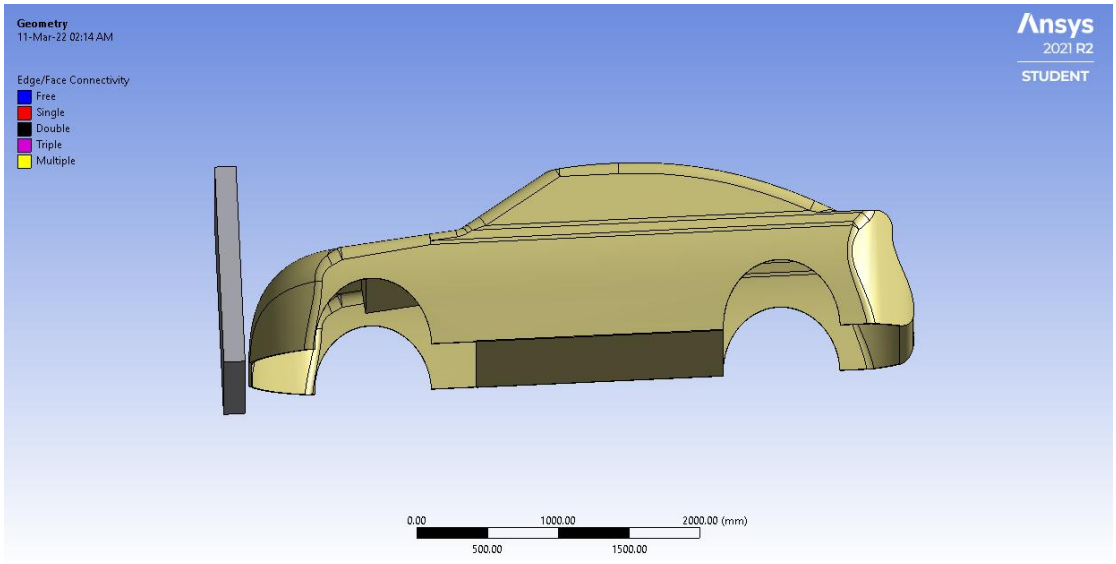


Fig 5.8 *Geometry representing car body and the wall*

Meshing: Triangles meshing method is used for the car body with a sizing of 100mm provided to the faces of the car. The wall is meshed using face sizing method.

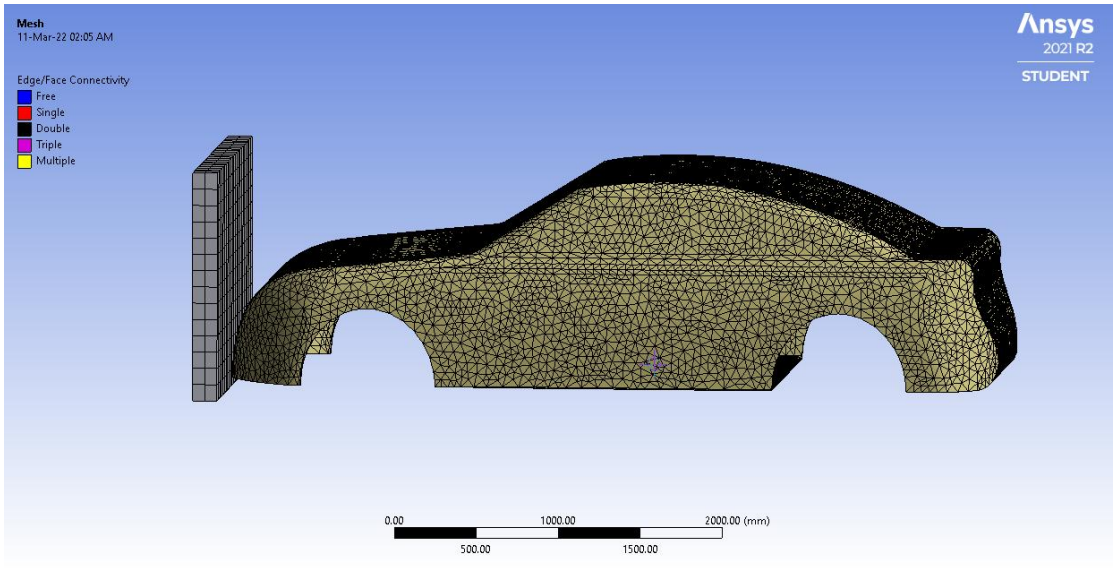


Fig 5.9 *Meshing of car body and the wall*

Boundary Conditions: To reduce the computational time, fixed support is provided to the back faces of the car and Wall is given a displacement of 500mm along the collision axis.

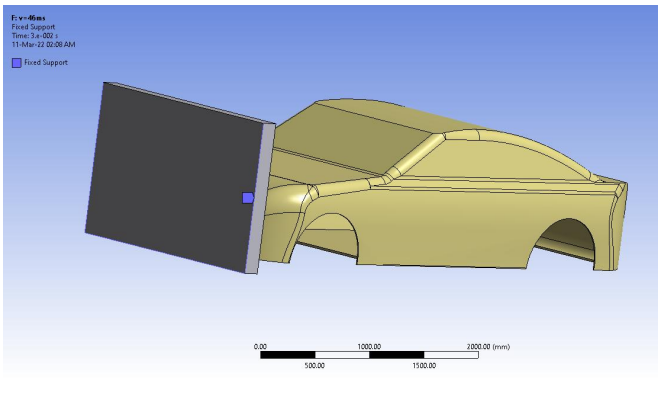


Fig 5.10 *Fixed support for car body*

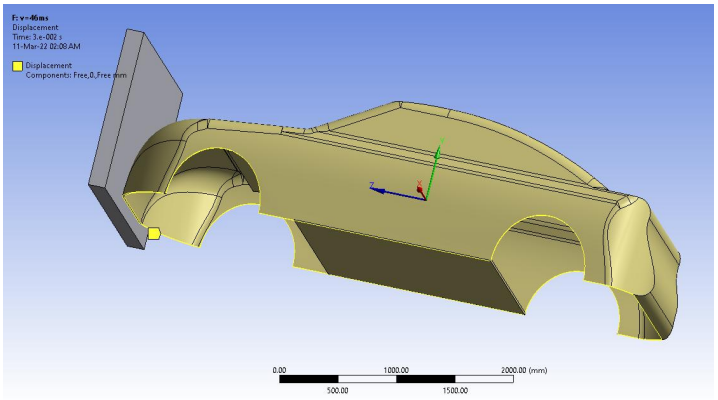


Fig 5.11 *displacement at the wall*

Analysis Settings:
End time – 0.03s
Initial, Minimum, Maximum time steps – Program Controlled.
Solution obtained:

Total Deformation –

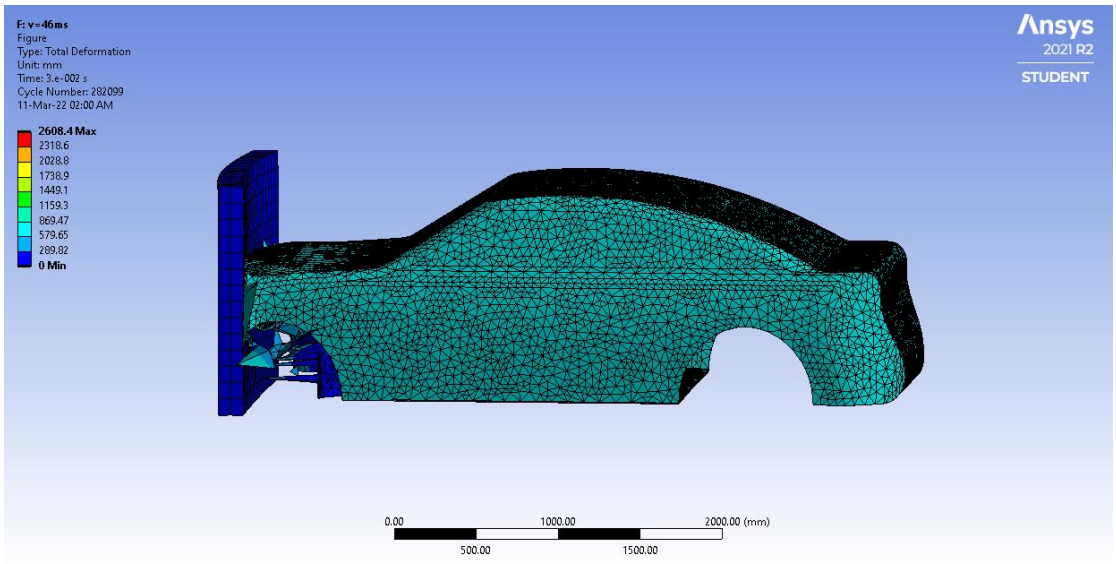


Fig 5.12 *Solution representing total deformation of the car body*

| | Time [s] | Minimum [mm] | Maximum [mm] | Average [mm] |
|---|---|---|---|---|
| 1 | 1.1755e-038 | 0. | 0. | 0. |
| 2 | 1.5e-003 | 0. | 73.779 | 65.699 |
| 3 | 3.e-003 | 0. | 149.97 | 129.17 |
| 4 | 4.5e-003 | 0. | 223.52 | 189.2 |
| 5 | 6.e-003 | 0. | 288.53 | 245.37 |
| 6 | 7.5001e-003 | 0. | 351.96 | 297.44 |
| 7 | 9.e-003 | 0. | 407.37 | 345.16 |
| 8 | 1.05e-002 | 0. | 464.8 | 388.95 |
| 9 | 1.2e-002 | 0. | 530.5 | 428.7 |
| 10 | 1.35e-002 | 0. | 599.04 | 464.27 |
| 11 | 1.5e-002 | 0. | 670.16 | 496.71 |
| 12 | 1.65e-002 | 0. | 752.1 | 526.7 |
| 13 | 1.8e-002 | 0. | 895.95 | 554.84 |
| 14 | 1.95e-002 | 0. | 1041.4 | 581.68 |
| 15 | 2.1e-002 | 0. | 1187.8 | 606.37 |
| 16 | 2.25e-002 | 0. | 1334.9 | 628.35 |
| 17 | 2.4e-002 | 0. | 1553.2 | 647.39 |
| 18 | 2.55e-002 | 0. | 1813.2 | 662. |
| 19 | 2.7e-002 | 0. | 2076.4 | 671.14 |
| 20 | 2.85e-002 | 0. | 2341.7 | 674.05 |
| 21 | 3.e-002 | 0. | 2608.4 | 671.79 |

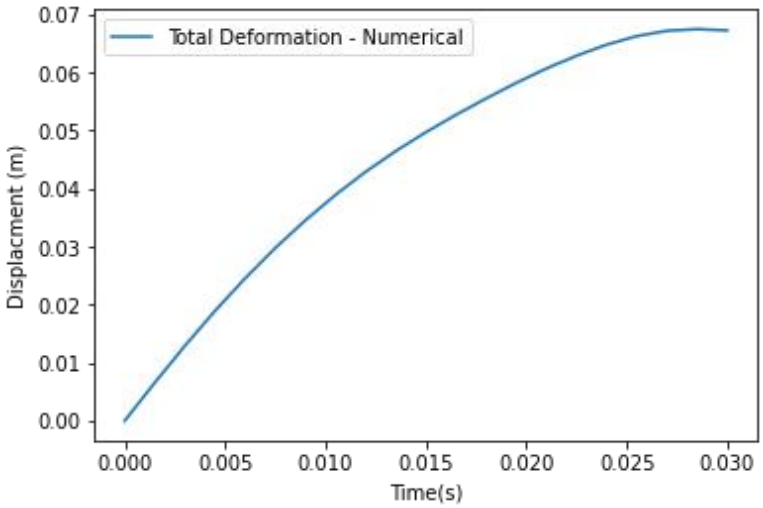


Fig 5.13 *Solution for total deformation*

Fig 5.14 *Plot between average deformation vs time*

The solution obtained from the numerical analysis are described as in fig 5.14. The deformation of the car increases throught the prescribed time as the crash is not completely simulated. The car crash is simulated for very small time as it is computationally expensive and also take several hours to simulate. In reality, the deformation should increase until a fixed time called as time of maximum dynamic crush, after which the car bounces back.

Equivalent Stress:

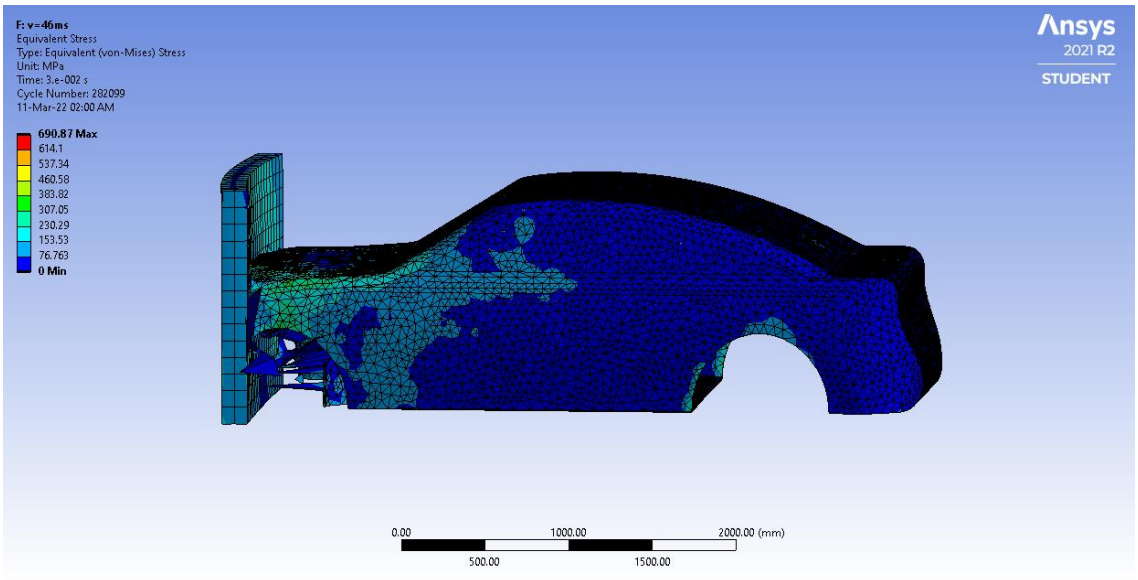


Fig 5.15 *Solution representing equivalent stress of the car body*

| | Time [s] | Minimum [MPa] | Maximum [MPa] | Average [MPa] |
|---|---|---|---|---|
| 1 | 1.1755e-038 | 0. | 0. | 0. |
| 2 | 1.5e-003 | 0. | 609.63 | 42.125 |
| 3 | 3.e-003 | 1.1522 | 631.99 | 80.518 |
| 4 | 4.5e-003 | 1.4269 | 642. | 92.316 |
| 5 | 6.e-003 | 2.1447 | 646.85 | 93.085 |
| 6 | 7.5001e-003 | 0. | 648.28 | 96.632 |
| 7 | 9.e-003 | 0. | 634.69 | 90.235 |
| 8 | 1.05e-002 | 0. | 575.39 | 83.229 |
| 9 | 1.2e-002 | 0. | 635.16 | 89.934 |
| 10 | 1.35e-002 | 0. | 619.99 | 78.47 |
| 11 | 1.5e-002 | 0. | 572.43 | 70.444 |
| 12 | 1.65e-002 | 0. | 608.74 | 71.289 |
| 13 | 1.8e-002 | 0. | 584.75 | 69.292 |
| 14 | 1.95e-002 | 0. | 594.46 | 68.879 |
| 15 | 2.1e-002 | 0. | 556.36 | 68.761 |
| 16 | 2.25e-002 | 0. | 658.28 | 91.778 |
| 17 | 2.4e-002 | 0. | 652.12 | 107.34 |
| 18 | 2.55e-002 | 0. | 669.52 | 100.1 |
| 19 | 2.7e-002 | 0. | 648.01 | 101.81 |
| 20 | 2.85e-002 | 0. | 686.99 | 82.721 |
| 21 | 3.e-002 | 0. | 690.87 | 73.41 |

Fig 5.16 *Solution for equivalent stress*

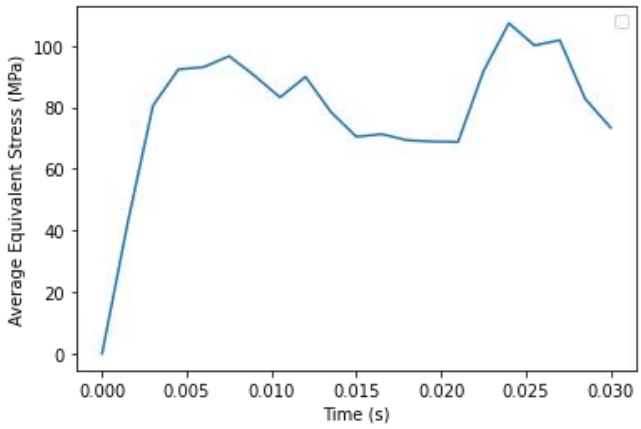


Fig 5.17 *Plot between stress and time*

The solution obtained from the numerical analysis are described as in fig 5.17. The maximum average stress obtained during 0.03s is 92MPa.

The below is the comparison between the solutions found using Finite Element Analysis, PINN techniques and analytical methods.

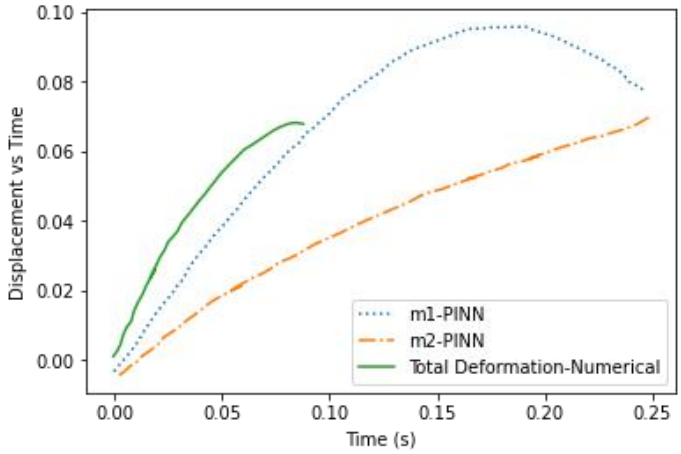


Fig 5.18 *Comparison between Numerical (FEM) and PINN methods.*

The comparison of the displacement of the car during the crash between Numerical and PINN methods is as shown in the above fig 5.18. Since, the car crash is simulated using Finite Element Methods for very less duration of the crash, the plot corresponding to Finite Element Analysis is shorter in time. There is a sight variation in the results of the two methods, this may be due to the inaccurate approximation of a car crash problem to a simple spring mass damper analytical model.

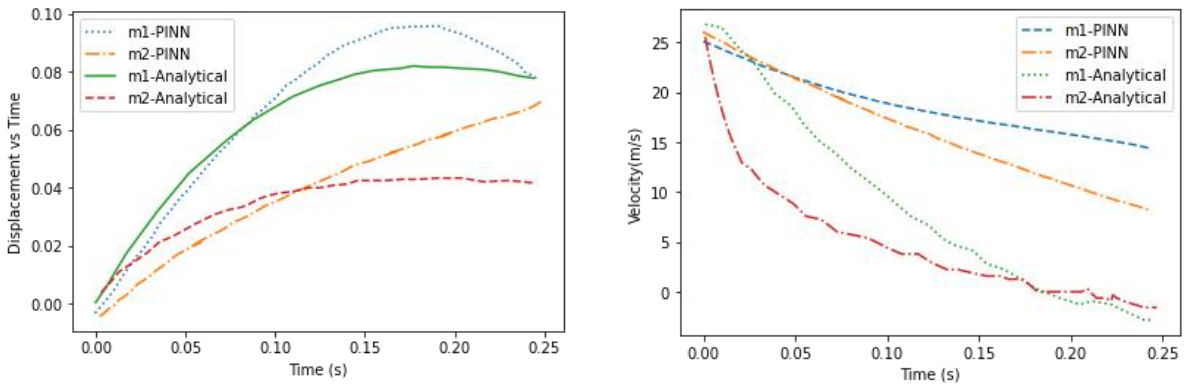


Fig 5.19 *Comparison of solution of 2 DOF between Analytical and Numerical (PINN) methods.*

The comparison of displacement and velocity of both $m_1$ and $m_2$ in the 2 DOF analytical model between the analytical model [2] and numerical PINN methods are shown in fig 5.19. There is a slight variation in the results though the behavior is almost similar in displacement, the velocity plots differ significantly. This may be due to the convergence issues in the weights

optimization task due to less boundary conditions. The boundary conditions imposed are highly insufficient for the neural network approximation. A better analytical model with better posed boundary conditions helps the accuracy of the results to improve very significantly.

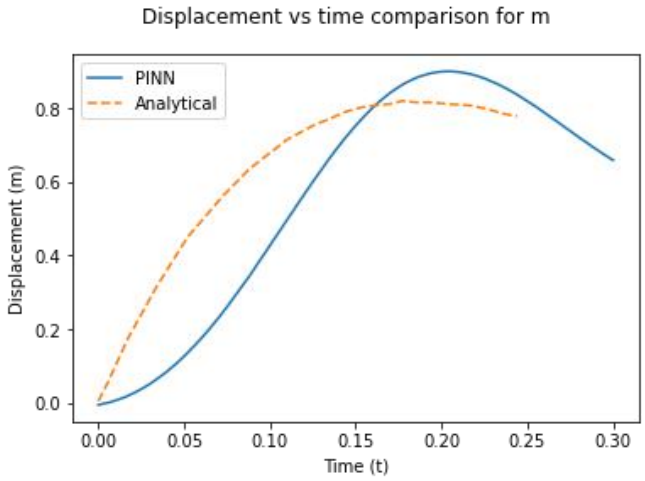


Fig 5.20 *Comparison of solution of 1 DOF between Analytical and Numerical (PINN) methods*

The fig 5.20 above depicts the comparison of the displacement of the mass in 1 DOF analtyical model predicted using PINN methods and analytical solution [2]. There is significantly large variation in the results of both methods. As discussed above, this may be due to the convergence issues in the weights optimization task due to less boundary conditions. The boundary conditions imposed are highly insufficient for the neural network approximation. A better analytical model with better posed boundary conditions helps the accuracy of the results to improve very significantly.

# 6. CONCLUSION

Car crash analysis is one of the main areas of research in automotive industries and solving it traditionally by wrecking the car to the wall physically causes a lot of financial loss. CAE solvers like ANSYS uses FEA to solve the problem but it is not computationally effective as the previous result found using explicit dynamics in ANSYS took almost 25 hours. Solving PDEs using Neural networks is a better and efficient way provided the availability of the best suitable analytical model.

In this project, the mathematical models of car crash are taken, whose impact responses are similar to free vibrational analysis of 1 degree of freedom and 2 degrees of freedom damped systems. Using the known structural parameters of the analytical model of car crash, the PDE solution is found by the PINN methods for both 1 DOF and 2 DOF models. However, an attempt to find the structural parameters using inverse PINN methods have been made. Later, the weights of the neural network trained for the above problems are directly used as an initial guess for a new similar problem with different structural parameters.

The variation in the results of the PINN methods and Finite Element Analysis may be due to the fact that one cannot mathematically model a car crash in terms of a simple 1 DOF or 2 DOF spring mass damper system. The variation in the results of the PINN methods and Analytical solution may be to the convergence issues in the weights optimization task due to less boundary conditions.

## REFERENCES


1. M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, 10.1016/j.jcp.2018.10.045.
2. Journal of Computational Physics, Munyazikwiye, Bernard.B. & Karimi, Hamid & Robbersmyr, Kjell. (2017). Optimization of Vehicle-to-Vehicle Frontal Crash Model Based on Measured Data Using Genetic Algorithm. IEEE Access. PP. 1-1. 10.1109/ACCESS.2017.2671357.
3. W. Pawlus, H. R. Karimi, and K. G. Robbersmyr, ‘‘Development of lumped-parameter mathematical models for a vehicle localized impact,’’ J. Mech. Sci. Technol., vol. 25, no. 7, pp. 1737–1747, 2011.
4. Raissi, Maziar & Perdikaris, Paris & Karniadakis, George. (2017). Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Differential Equations.
5. Query vehicle crash test database on select test parameters, National Highway Traffic Safety Administration, Available: http://www-nrd.nhtsa.gov/database/vsr/veh/ querytest.aspx
6. Kirill Zubov, Zoe McCarthy, NeuralPDE: Automating Physics-Informed Neural Networks with error approximations, arXiv:2107.09443v1

## Python Code for solving 1 DOF model using PINN

```
# Import TensorFlow and NumPy
import tensorflow as tf
import numpy as np

# Set data type
DTYPE='float32'
tf.keras.backend.set_floatx(DTYPE)
# Set constants
m = 291
c = 13687
k = 45929

# Define initial condition
def fun_u_0(t):
    n = t.shape[0]
    return tf.zeros((n,1), dtype=DTYPE)

# Define residual of the PDE
def fun_r(t, x, x_t, x_tt):
    return m*x_tt + c*x_t + k*x

# Set number of data points
N_0 = 50
N_r = 10000

tmin = 0
tmax = 0.3
# Lower bounds
lb = tf.constant(tmin, dtype=DTYPE)
# Upper bounds
ub = tf.constant(tmax, dtype=DTYPE)
tf.random.set_seed(0)

# Draw uniform sample points for initial boundary data
t_0 = tf.ones((N_0,1), dtype=DTYPE)*lb
X_0 = tf.concat(t_0, axis=0)

# Evaluate intitial condition at x_0
u_0 = fun_u_0(t_0)

# Draw uniformly sampled collocation points
t_r = tf.random.uniform((N_r,1), lb, ub, dtype=DTYPE)

# Collect inital data in lists
t_data = [X_0]
u_data = [u_0]

def init_model(num_hidden_layers=2, num_neurons_per_layer=5):
    # Initialize a feedforward neural network
    model = tf.keras.Sequential()

    model.add(tf.keras.Input(shape=(1,)))
    scaling_layer = tf.keras.layers.Lambda(
                lambda x: 2.0*(x - lb)/(ub - lb) - 1.0)
    model.add(scaling_layer)
    for _ in range(num_hidden_layers):
        model.add(tf.keras.layers.Dense(num_neurons_per_layer,
            activation=tf.keras.activations.get('tanh'),
```

```
            kernel_initializer='glorot_normal'))
    model.add(tf.keras.layers.Dense(1))

    return model

def get_r(model, t_r):
    # A tf.GradientTape is used to compute derivatives in TensorFlow
    with tf.GradientTape(persistent=True) as tape:
            t = t_r
        tape.watch(t)
        u = model(t)
        u_t = tape.gradient(u, t)
        u_tt = tape.gradient(u_t, t)
    del tape
    return fun_r(t, u, u_t, u_tt)**2

def compute_loss(model, t_r, t_data, u_data):
    r = get_r(model, t_r)
    phi_r = tf.reduce_mean(tf.square(r))

    # Initialize loss
    loss = phi_r
    for i in range(len(t_data)):
        u_pred = model(t_data[i])
        loss += tf.reduce_mean(tf.square(u_data[i] - u_pred))

    return loss

def get_grad(model, t_r, t_data, u_data):

    with tf.GradientTape(persistent=True) as tape:
        tape.watch(model.trainable_variables)
        loss = compute_loss (model, t_r, t_data, u_data)

    g = tape.gradient(loss, model.trainable_variables)
    del tape
    return loss, g

model = init_model()
lr = tf.keras.optimizers.schedules.PiecewiseConstantDecay([1000,3000],[1e-2,1e-3,5e-4])
optim = tf.keras.optimizers.Adam(learning_rate=lr)
from time import time

#@tf.function
def train_step():
    loss, grad_theta = get_grad(model, t_r, t_data, u_data)

    # Perform gradient descent step
    optim.apply_gradients(zip(grad_theta, model.trainable_variables))

    return loss
# Number of training epochs
N = 100
hist = []
# Start timer
t0 = time()
for i in range(N+1):

    loss = train_step()
    # Append current loss to hist
```

```
    hist.append(loss.numpy())

    # Output current loss after 50 iterates
    if i%50 == 0:
        print('It {:05d}: loss = {:10.8e}'.format(i,loss))

# Print computation time
print('\nComputation time: {} seconds'.format(time()-t0))
import matplotlib.pyplot as plt
# Set up meshgrid
N = 600
tspace = np.linspace(lb, ub, N)
# Determine predictions
output= model(tf.cast(tspace,DTYPE))
uspace = tf.reshape(output, 600)
u1space = np.gradient(uspace, tspace)
u11space = np.gradient(u1space, tspace)
```

## Python code for solving 2 DOF model using PINN

```
# Import TensorFlow and NumPy
import tensorflow as tf
import numpy as np

# Set data type
DTYPE='float32'
tf.keras.backend.set_floatx(DTYPE)

# Set constants
m1 = 291
m2 = 582
c1 = 13687
c2 = 9952
k1 = 45929
k2 = 40731

# Define initial condition
def fun_u_0(t):
    n = t.shape[0]
    return tf.zeros((n,1), dtype=DTYPE)

# Define residual of the PDE
def fun_r_1(t, x, x_t, x_tt, y, y_t, y_tt):
    return m1*x_tt + (c1+c2)*x_t + (k1+k2)*x - c2*y_t -k2*y

def fun_r_2(t, x, x_t, x_tt, y, y_t, y_tt):
    return m2*y_tt + c2*y_t - c1*x_t + k2*y - k2*x

# Set number of data points
N_0 = 50
N_r = 10000

# Set boundary
tmin = 0
tmax = 0.3

# Lower bounds
lb = tf.constant(tmin, dtype=DTYPE)
# Upper bounds
ub = tf.constant(tmax, dtype=DTYPE)
# Set random seed for reproducible results
tf.random.set_seed(0)

# Draw uniform sample points for initial boundary data
t_0 = tf.ones((N_0,1), dtype=DTYPE)*lb
X_0 = tf.concat(t_0, axis=0)

# Evaluate intitial condition at x_0
u_0 = fun_u_0(t_0)

# Draw uniformly sampled collocation points
t_r = tf.random.uniform((N_r,1), lb, ub, dtype=DTYPE)

# Collect inital data in lists
t_data = [X_0]
u_data = [u_0]

def init_model(num_hidden_layers=2, num_neurons_per_layer=5):
```

```
    # Initialize a feedforward neural network
    model = tf.keras.Sequential()
    model.add(tf.keras.Input(shape=(1,)))
    scaling_layer = tf.keras.layers.Lambda(
                lambda x: 2.0*(x - lb)/(ub - lb) - 1.0)
    model.add(scaling_layer)
    for _ in range(num_hidden_layers):
        model.add(tf.keras.layers.Dense(num_neurons_per_layer,
            activation=tf.keras.activations.get('tanh'),
            kernel_initializer='glorot_normal'))
    # Output is two-dimensional
    model.add(tf.keras.layers.Dense(2))

    return model

def get_r(model, t_r):

    # A tf.GradientTape is used to compute derivatives in TensorFlow
    with tf.GradientTape(persistent=True) as tape:
        t = t_r
        tape.watch(t)
        u = model(t)[0]
        v = model(t)[1]
        u_t = tape.gradient(u, t)
        u_tt = tape.gradient(u_t, t)
        v_t = tape.gradient(v, t)
        v_tt = tape.gradient(v_t, t)
    del tape
    return (fun_r_1(t, u, u_t, u_tt, v, v_t, v_tt))**2 + (fun_r_2(t, u, u_t, u_tt, v, v_t, v_t
t))**2


def compute_loss(model, t_r, t_data, u_data):
    r = get_r(model, t_r)
    phi_r = tf.reduce_mean(tf.square(r))

    # Initialize loss
    loss = phi_r
    for i in range(len(t_data)):
        u_pred = model(t_data[i])
        loss += tf.reduce_mean(tf.square(u_data[i] - u_pred))

    return loss

def get_grad(model, t_r, t_data, u_data):

    with tf.GradientTape(persistent=True) as tape:
        tape.watch(model.trainable_variables)
        loss = compute_loss (model, t_r, t_data, u_data)

    g = tape.gradient(loss, model.trainable_variables)
    del tape
    return loss, g

model = init_model()
lr = tf.keras.optimizers.schedules.PiecewiseConstantDecay([1000,3000],[1e-2,1e-3,5e-4])
optim = tf.keras.optimizers.Adam(learning_rate=lr)
```

```
from time import time

@tf.function
def train_step():
    loss, grad_theta = get_grad(model, t_r, t_data, u_data)

    # Perform gradient descent step
    optim.apply_gradients(zip(grad_theta, model.trainable_variables))

    return loss
# Number of training epochs
N = 100
hist = []
# Start timer
t0 = time()
for i in range(N+1):

    loss = train_step()

    # Append current loss to hist
    hist.append(loss.numpy())
    if i%50 == 0:
        print('It {:05d}: loss = {:10.8e}'.format(i,loss))

# Print computation time
print('\nComputation time: {} seconds'.format(time()-t0))
# Set up meshgrid
N = 600
tspace = np.linspace(lb, ub, N)
# Determine predictions
output= model(tf.cast(tspace,DTYPE))
uspace = output[:, 0:1]
vspace = output[:, 1:2]
uspace = tf.reshape(output[:, 0:1], 600)
u1space = np.gradient(uspace, tspace)
u11space = np.gradient(u1space, tspace)
vspace = tf.reshape(output[:, 1:2], 600)
v1space = np.gradient(uspace, tspace)
v11space = np.gradient(u1space, tspace)
```